



ENME 441

05 – ESP32 Input and Output

Topics



PART 1

GPIO pins

Digital in and out

- machine.Pin API
- Which C3 pins are usable



PART 2

pwm

Pseudo-analog outputs

- Creating a pwm object
- Duty cycle and frequency



PART 3

interrupts

Reacting without polling

- Callbacks and IRQs
- One handler, many pins
- Limitations on concurrency



PART 4

adc

Reading an analog voltage

- 12-bit conversion
- Attenuation ranges

01

GPIO pins

Digital in and out



machine.Pin

A machine.Pin object by itself supports digital input and output

```
from machine import Pin

# Inputs:
p1 = Pin(p, Pin.IN)           # set up pin as input
p2 = Pin(p, Pin.IN, Pin.PULL_UP) # apply pull-up resistor
p3 = Pin(p, Pin.IN, Pin.PULL_DOWN) # apply pull-down resistor

value = p1.value()           # read digital value on the Pin

# Outputs:
p4 = Pin(p, Pin.OUT)         # set up pin as output
p5 = Pin(p, Pin.OUT, value=1) # start with output (0|1)

p4.value(1)                  # set Pin high
p4.on()                      # set Pin high (alternate)

p4.value(0)                  # set Pin low
p4.off()                     # set Pin low (alternate)
```

- **value() both reads and writes**

Called with no argument it returns the pin state; called with 0 or 1 it sets it. Which one you get depends on the argument, not the pin.

- **A floating input reads garbage**

An input wired to nothing picks up whatever charge is nearby. PULL_UP or PULL_DOWN ties the pin to either 3.3V or 0V through an internal resistor.

- **on() and off() are shortcut aliases**

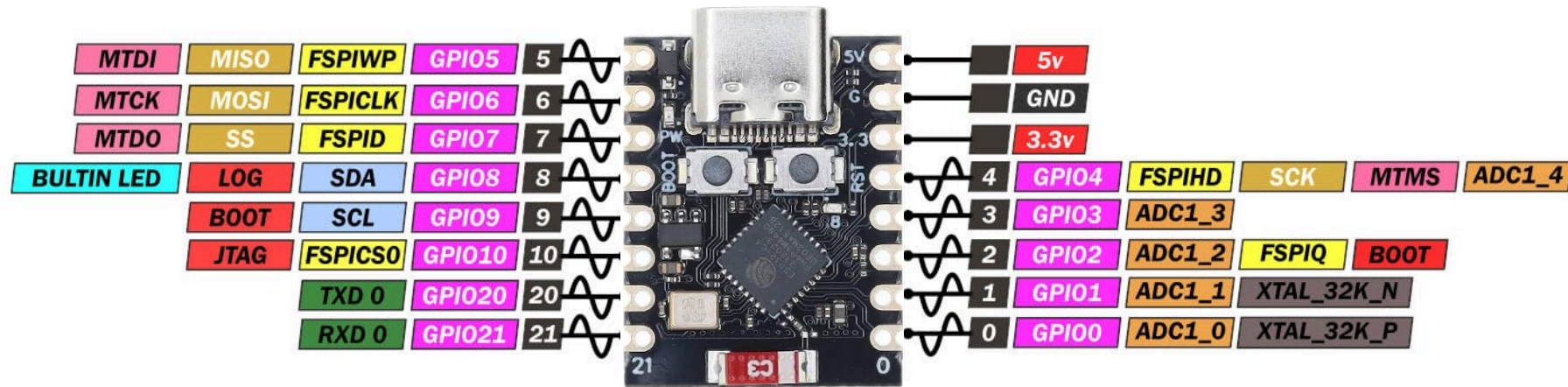
Same as value(1) and value(0)

- **See docs.micropython.org for details**

[library/machine.Pin.html](#) has the full argument list, including drive strength and open-drain modes.

Available GPIO Pins

13 GPIO pins, 5 with constraints.



GPIO 0 – 5

Free for anything. 0–4 are also the ADC1 inputs.



GPIO 2, 8, 9

Boot straps — must read HIGH at reset. Never pull these down, or the board will not start.



GPIO 8

Built-in LED, wired through a pull-up: 0 is ON, 1 is OFF. Also a boot strap.



GPIO 20, 21

Default UART0 TX and RX. Usable as GPIO, but you lose the serial port on those pins.

More importantly, these pins sit next to the RF antenna — connections can detune the antenna and degrade Wi-Fi signal range and stability!

On-Chip Sensors

While the classic ESP32 has 3 built-in sensors, the C3 only has internal temperature sensing



Internal temperature sensor

Measures the die, not the room — it typically reads several degrees Celsius above ambient when the WiFi radio has been busy. Useful for thermal sanity checks, not accurate ambient sensing.

```
import esp32

esp32.mcu_temperature()    # degrees C, on the C3

# On the classic ESP32 the same reading is
# esp32.raw_temperature(), and it returns
# degrees FAHRENHEIT. Different name,
# different units, same idea.
```

Any code you find online that calls `hall_sensor()` or `TouchPad()` will raise an `AttributeError` on the C3.

GONE ON THE C3



Hall effect sensor

`esp32.hall_sensor()` has been removed from MicroPython entirely, and the C3 has no Hall sensor to read in the first place.



Capacitive touch

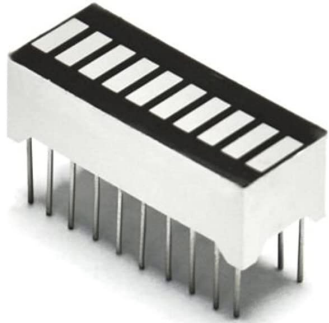
`machine.TouchPad` does not exist in C3 firmware. There is no touch peripheral on this chip.

Blink an External LED Array

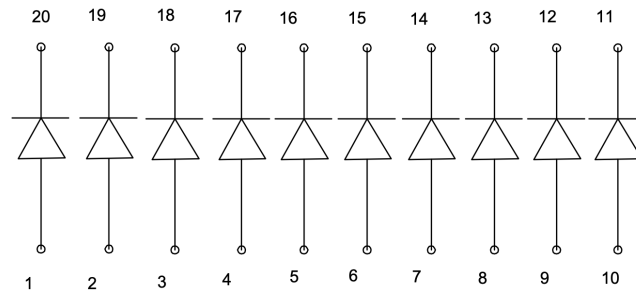


Try it on your own hardware

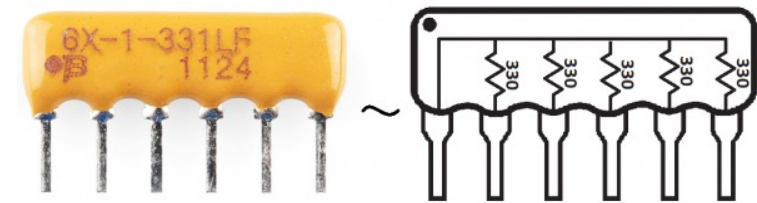
Wire up a 10-segment LED bar and 10-element network resistor to your ESP32 as shown below. Connect 2 GPIO pins to drive a pair of LEDs.



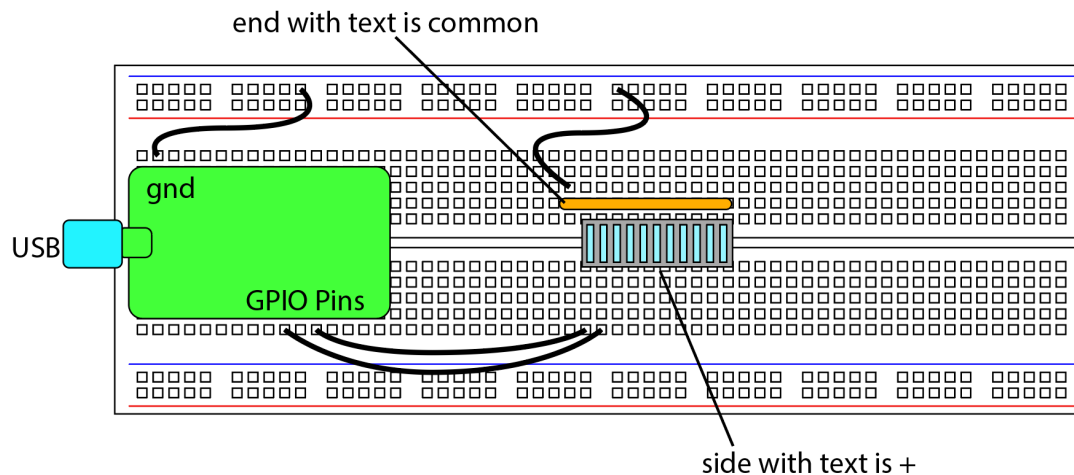
10-segment LED bar



Pin numbering — one LED per segment



Network resistor (330 Ω) — six resistors sharing one common pin



Never drive an LED bare

A GPIO pin driving an LED with no series resistor can push more current than either part is rated for. The network resistor puts an independent resistor in multiple legs at once.

Watch polarity: the LED bar marked side is the anode, and the network resistor's marked end is its common pin.



blink.py

The "hello world" of physical computing — write it from scratch.

```
# blink.py
#
# Blink an LED on and off at 1 Hz
# ESP32 C3
```

```
from machine import Pin
from time import sleep
```

```
led = Pin(0, Pin.OUT)
```

```
try:
    while True:
        led.on()
        sleep(0.5)
        led.off()
        sleep(0.5)
```

```
except KeyboardInterrupt:
    print('Exiting\n')
```

- **0.5s on + 0.5s off (1 Hz)**

One complete on-off cycle per second. The two sleeps add up to the period ($T = 1/f$).

- **Wrap the loop in try / except**

Without it, ctrl-C leaves the pin in whatever state it happened to be in. With it, you get a clean exit and a message.

- **Swap Pin(0) for Pin(8) to use the built-in LED**

Remember it is wired through a pull-up, so on() and off() come out backwards on that pin.



blink_pair.py

Now extend to driving 2 LEDs at the same time

BRUTE FORCE APPROACH

```
# blink_pair.py
#
# Blink 2 LEDs on and off at 1 Hz, with opposite polarity
# ESP32 C3

from machine import Pin
from time import sleep

led1 = Pin(0, Pin.OUT)
led2 = Pin(1, Pin.OUT)

try:
    while True:
        led1.on()
        led2.off()
        sleep(0.5)
        led1.off()
        led2.on()
        sleep(0.5)

except KeyboardInterrupt:
    print('Exiting\n')
```

BETTER STYLE: USE A FLAG

```
# blink_pair_flag.py
#
# Blink 2 LEDs on and off at 1 Hz, with opposite polarity
# ESP32 C3

from machine import Pin
from time import sleep

led1 = Pin(0, Pin.OUT)
led2 = Pin(1, Pin.OUT)
state = 1      # flag for led1 ON (and led2 OFF)

try:
    while True:
        led1.value(state)
        led2.value(~state)  # could use `not state` instead
        sleep(0.5)
        state = ~state      # flip the flag

except KeyboardInterrupt:
    print('Exiting\n')
```



Blink using toggle()

An even easier solution is to use the `Pin.toggle()` method, which flips the current Pin state directly. In this case, be sure to set an initial value when declaring each Pin object.

EASIEST APPROACH – USE `Pin.toggle()`

```
# blink_pair_toggle.py
#
# Blink 2 LEDs on and off at 1 Hz, with opposite polarity
# ESP32 C3

from machine import Pin
from time import sleep

led1 = Pin(0, Pin.OUT, value=0)  # Set initial value
led2 = Pin(1, Pin.OUT, value=1)

try:
    while True:
        led1.toggle()
        led2.toggle()
        sleep(0.5)

except KeyboardInterrupt:
    print('Exiting\n')
```



digital_in.py

Read the voltage on a Pin as a digital input

```
# digital_in.py
#
# Read digital input values. Connect switches (or wires) between
# GPIO pins and defined voltage levels (0V or 3.3V)
#
# The ESP32 C3 GPIO pins are NOT 5V tolerant -- the absolute
# maximum is 3.6V. Driving a pin from 5V will damage the chip.
#
# ESP32 C3

from machine import Pin
from time import sleep

p1 = Pin(5, Pin.IN)
p2 = Pin(6, Pin.IN, Pin.PULL_UP)
p3 = Pin(7, Pin.IN, Pin.PULL_DOWN)

while True:
    print(f'{p1.value() }, {p2.value() }, {p3.value() }')
    sleep(0.1)
```

- **Run it without jumpers**

p1 is floating and will flicker between 0 and 1 as you move your hand near the board.
p2 and p3 will sit steady at 1 and 0, respectively, due to the pull-up and pull-down resistors

- **Jumper pin2 to ground**

The pulled-up pin will go to 1

- **Jumper pin3 to 3.3V**

The pulled-down pin will go to 1

- **3.3 V, never 5 V**

C3 GPIO are not 5 V tolerant — 3.6 V is the absolute maximum. A 5 V logic signal wired straight to a pin will damage the chip.

02

pwm

Pulse width modulation to generate pseudo-analog outputs



Pulse Width Modulation

There is no digital-to-analog conversion (DAC) on the C3. The classic ESP32 has two (true) analog outputs, but the C3 chip has none.

However, switch a digital output “fast enough” and the output signal becomes analog. The definition of “fast enough” depends on the corner frequency of the circuit being driven, and the level of ripple that can be tolerated in the output signal.

1

Two knobs: frequency & duty cycle

```
my_pwm = PWM(PIN, freq=FREQ_HZ, duty=DC_10_BIT)
```

2

Circuit corner frequency dictates minimum PWM frequency

3

Three ways to set the duty cycle

`duty()` takes a 10-bit value: 0–1023

`duty_u16()` takes a 16-bit value: 0–65535

`duty_ns()` takes the HIGH time for a given period ($1/\text{freq}$) in nanoseconds.

ESP32-C3 LIMITS

Any GPIO can use PWM

- PWM is routed through the GPIO matrix, so any pin can carry it.

6 channels

- Six PWM outputs active at once (other ESP32 variants have 8 or 16).

4 timers

- Up to 4 distinct frequencies at once. Channels sharing a timer share its frequency.

14-bit hardware

- The API takes a 16-bit duty value and scales it down; the C3's LEDC hardware resolves 14 bits.



pwm_blink.py

The same blink as before, but the hardware does the timing instead of sleep().

```
# pwm_blink.py
#
# Basic PWM example
# ESP32 C3 SuperMini

from machine import Pin, PWM

# Use PWM on built-in LED
pwm = PWM(Pin(8), freq=10, duty=512)

f = pwm.freq()    # get the base frequency; pwm.freq(x) sets it.
                  # Channels sharing an LEDC timer share a
                  # frequency, so changing one can move another.

# Alternate ways to set the duty cycle to 50% (for example):
# pwm.duty(512)           # set 10-bit duty cycle (0-1023)
# pwm.duty_u16(32768)     # set 16-bit duty cycle (0-65535)
# pwm.duty_ns(int(1e9/f/2)) # set pulse width in nanoseconds

try:
    while True:
        pass

except KeyboardInterrupt:
    pwm.deinit()    # Turn off the PWM
```

- **The empty while loop is the point**

Once PWM is configured the peripheral runs on its own. The loop exists only to keep the script alive — the CPU does nothing.

- **Three ways to say the same thing**

duty() takes 0–1023, duty_u16() takes 0–65535, duty_ns() takes the HIGH time directly. Pick one and stay with it.

- **10 Hz is visible; 5 kHz is not**

At this frequency you see the LED blink. Raise it past about 100 Hz and your eye averages it into steady brightness instead.

- **deinit() releases the channel**

Six channels is not many. Leave one allocated and the next PWM you create may fail for no visible reason.



pwm_pulse.py

Sweep the duty cycle instead of holding it, and the LED breathes.

SET UP THE CHANNEL

```
# pwm_pulse.py
#
# Pulse an LED via PWM
#
# ESP32 C3 SuperMini

from machine import Pin, PWM
from time import sleep

pwm = PWM(Pin(8))    # Use the built-in blue LED
                    # 5 kHz base frequency by default

# Alternates:
# pwm = PWM(Pin(8), freq=5000, duty=0)
# pwm.init(freq=5000, duty=0)
# pwm.freq(5000)
# pwm.duty(0)
```

SWEEP THE DUTY CYCLE

```
def pulse():

    while True:
        for i in range(1024):
            pwm.duty(i)
            sleep(0.001)
        for i in range(1023,0,-1):
            pwm.duty(i)
            sleep(0.001)

    try:
        pulse()

    except KeyboardInterrupt:
        pwm.deinit()    # Turn off the PWM
```

Two sweeps, up then down: `range(1024)` climbs and `range(1023,0,-1)` descends, so one full cycle takes about two seconds at 1 ms per step. Unlike `pwm_blink.py`, this one blocks — the CPU sits in that loop the whole time and nothing else in your program runs. The next slide is how you get around that.



pwm_pulser.py

Wrap the sweep in a class and run it on its own thread, so the rest of your program keeps going.

Pulser CLASS

```
from machine import PWM, Pin
from time import sleep
from _thread import start_new_thread

class Pulser:

    def __init__(self, pin, period = 1.0):
        self.pwm = PWM(pin)
        self.isOn = False
        self.period = period

    def start(self):
        self.isOn = True
        start_new_thread(self.__pulse, ())

    def stop(self):
        self.isOn = False
```

```
    def __pulse(self):
        steps = 100
        while self.isOn:
            for i in range(steps):
                dc = int(2**16 * i / steps)
                self.pwm.duty_u16(dc)
                sleep(self.period / 2 / steps)
            for i in range(steps-1, 0, -1):
                dc = int(2**16 * i / steps)
                self.pwm.duty_u16(dc)
                sleep(self.period / 2 / steps)
```

```
p = Pulser(pin = 8, period = 1.0)
```

```
for i in range(3):
    print('Pulser ON')
    p.start()
    sleep(5)
    print('Pulser OFF')
    p.stop()
    sleep(5)
```

Note: We will talk more about threads and concurrency later in the class.

03

Interrupt Requests (IRQs)



Launching a threaded callback function on a hardware input signal

Threaded Callbacks via Interrupt Requests

The hardware to run specified code when a pin state changes, regardless of what else is happening in the main code. This is part of the concept of **concurrency** which will be discussed in more detail later in the class.

1

A thread is code that runs independently of the main code

It has its own flow, and does not wait for your main code which can be doing other things while the thread runs.

2

A callback is a function you hand to something else

You pass the function itself. Something else decides when to "call back" and run it.

3

A threaded callback is triggered by an IRQ

When a pin state change raises an interrupt request, MicroPython runs the callback function in a new thread



The handler always gets the Pin

The callback is automatically passed the Pin object (not the pin number!) that raised the request. This lets a single function serving several pins know which pin triggered the callback.



wake= does not work on the C3

It needs the RTC IO subsystem, which the C3 does not have. Use `esp32.wake_on_gpio()` to wake from sleep instead.

```
Pin.irq(handler=callback_function,
        trigger=(Pin.IRQ_FALLING | Pin.IRQ_RISING),
        wake=(machine.IDLE | machine.SLEEP | machine.DEEPSLEEP))
```

Threaded Callback Limitations



One callback per Pin object

Assigning a second handler to the same pin replaces the first — it does not add to it.



Keep handlers short

A callback that sleeps or loops blocks every other interrupt on the board for as long as it runs.



All callbacks share a single thread

Two IRQs firing close together are handled one after the other, never at the same time.



A callback cannot run beside your threads

It will not overlap a thread you started yourself either. There is no multiprocessing in MicroPython to work around it.

callbacks.py

Two pins, one handler, two different trigger conditions.

DEFINE THE HANDLER

```
# External interrupts using callback functions
# ESP32-C3

from machine import Pin

# Define input pins:
in1 = Pin(0, Pin.IN, Pin.PULL_DOWN)
in2 = Pin(1, Pin.IN, Pin.PULL_DOWN)

# Define a callback function:
def myCallback(p):
    print(f'Transition detected on {p}')

# Pin.irq(handler=Function_to_call,
#         trigger=(Pin.IRQ_FALLING | Pin.IRQ_RISING),
#         wake=(machine.IDLE | machine.SLEEP |
#               machine.DEEPSLEEP))
```

ATTACH IT TO THE PINS

```
# Note: the wake argument is not supported on the ESP32-C3,
# which has no RTC IO subsystem -- passing it raises
# "invalid pin for wake". Use esp32.wake_on_gpio() instead.

# Execute myCallback() if pin 1 goes HIGH:
in1.irq(handler=myCallback, trigger=Pin.IRQ_RISING)

# Execute myCallback() if pin 2 goes either LOW or HIGH:
in2.irq(handler=myCallback,
        trigger=(Pin.IRQ_FALLING | Pin.IRQ_RISING))

try:
    while True:
        pass
except:
    print('Exiting\n')
```

IRQ_RISING fires once, on the 0 → 1 edge; in2 OR's both edges together, so it reports the release as well as the press. Touch a jumper to 3.3 V and expect several prints rather than one — mechanical contact bounces, and every bounce is a real edge. Debouncing is a later lecture; for now, just expect it.



blink_callback_dictionary.py

One handler for two pin pairs, using a dictionary to track the input/output pin relationships.

MAP INPUTS TO OUTPUTS

```
# blink_callback_dictionary.py
# ESP32-C3
#
# Use a dictionary to relate two input and output pins.
# Flip the state of an output pin when the
# corresponding input pin sees a transition.

from machine import Pin

# Define input pins:
in1 = Pin(0, Pin.IN)
in2 = Pin(1, Pin.IN)

# Define output pins:
out1 = Pin(2, Pin.OUT)
out2 = Pin(3, Pin.OUT)

# relate input/output pins using a dictionary:
pins = {in1:out1, in2:out2}
```

LOOK UP, THEN FLIP

```
# Define a callback function:
def myCallback(p) :
    print(f'Transition detected on pin {p}')
    pin = pins[p] # find the output Pin from the dictionary
    pin.value(int(not pin.value())) # flip the pin state

# Execute myCallback() if pin 1 goes HIGH:
in1.irq(handler=myCallback, trigger=Pin.IRQ_RISING)

# Execute myCallback() if pin 2 goes either LOW or HIGH:
in2.irq(handler=myCallback,
        trigger=(Pin.IRQ_FALLING | Pin.IRQ_RISING))

try:
    while True:
        pass
except KeyboardInterrupt:
    print('Exiting\n')
```

Pin objects are usable as dictionary keys, and Pin(0) always returns the same object, so p handed to the callback is the same key in the dictionary. This is the pattern to use when one handler serves many pins (rather than writing a chain of `if p is in1` tests).

04

ADC

Analog-to-digital conversion - reading an analog voltage



Analog-to-Digital Conversion

1

Keep all analog inputs on GPIO 0 – 4

The C3 has 2 ADC blocks, but only one is available when using WiFi



ADC1

GPIO 0, 1, 2, 3, 4

Five channels, always available.



ADC2

GPIO 5

One channel, and it shares hardware with the WiFi radio. Readings are unreliable when the radio is on.

ESP32-C3 ADC properties

12-bit

`read()` returns 0 – 4095

16-bit

`read_u16()` rescales to 0 – 65535

microvolts

`read_uv()` applies the calibration

Attenuation and Range

The ADC circuit only spans ~0.75 V input range. Use attenuation of the input signal to expand the range.

ATTENUATION	SETTING	INPUT RANGE (ESP32-C3)
No attenuation	<code>ADC.ATTN_0DB</code>	0 mV ~ 750 mV
2.5 dB	<code>ADC.ATTN_2_5DB</code>	0 mV ~ 1050 mV
6 dB	<code>ADC.ATTN_6DB</code>	0 mV ~ 1300 mV
11 dB	<code>ADC.ATTN_11DB</code>	0 mV ~ 2500 mV

These are the C3's numbers

Every ESP32 variant has a different table of input range values.

Even at 11 dB you cannot reach 3.3 V

The top of the range is about 2.5 V. Divide a larger signal down with two resistors before it reaches the pin.

Linearity degrades at both ends

Readings near 0 and the top of the range are non-linear. Keep your desired signal in the middle of the range when possible.



adc.py

Set up a potentiometer between 3.3 V and ground, with the wiper on GPIO 0.

SET UP THE CONVERTER

```
# adc.py
#
# Analog-to-digital conversion example
# ESP32 C3

from machine import Pin, ADC
from time import sleep

# ESP32 has 12-bit ADC

# ADC is available on pins 0-4 on the ESP32-C3.
# These are the ADC1 channels. GPIO 5 is the only ADC2 channel,
# and ADC2 shares hardware with the radio -- readings there are
# unreliable whenever WiFi is active, so stay on pins 0-4.

# Set up a potentiometer with outer pins at 3.3V and gnd,
# and the middle pin connected to GPIO 0

# create an ADC object
adc = ADC(Pin(0))      # options for C3: 0,1,2,3,4
```

THEN READ IT THREE WAYS

```
# Add attenuation to increase the input voltage range
# (ranges below are for the ESP32-C3 -- other ESP32 variants have
# different values):
# ADC.ATTN_0DB: No attenuation (0mV ~ 750mV)
# ADC.ATTN_2_5DB: 2.5dB attenuation (0mV ~ 1050mV)
# ADC.ATTN_6DB: 6dB attenuation (0mV ~ 1300mV)
# ADC.ATTN_11DB: 11dB attenuation (0mV ~ 2500mV)

adc.atten(ADC.ATTN_11DB)

while True:
    v1 = adc.read()          # 0 to (2^12)-1 = 4095
    v2 = adc.read_u16()      # 0 to (2^16)-1 = 65535
    v3 = adc.read_uv()       # microvolts w/o attenuation
    print(f'read: {v1:4.0d}, read_u16: {v2:5.0d}, read_uv: {v3:7.0d}')
    sleep(0.1)
```

Turn the pot end to end and watch the values change.
`read()` and `read_u16()` are the same measurement scaled differently
`read_uv()` yields the actual voltage in units of μV .

LAB ASSIGNMENT

Lab 4: PWM and IRQs



Next steps



Shift registers

Increase available GPIO pins and extend current handling limits for output pins using one or more shift registers.



Concurrency

Extending beyond IRQs using the `_thread` and `asyncio` modules to write code that can run concurrently with the main thread.